

Week 3: Perform Exploratory Data Analysis

Project: Airline Passenger Demand Forecasting using BTS T-100 Domestic Segment (All Carriers)

Week 3 Objectives

- Split model-eligible observations into chronological training, validation, and test partitions.
- Conduct exploratory data analysis **only on training partition**.
- Identify data-quality and modeling issues.
- Document preprocessing recommendations for subsequent modeling weeks.

Project predicts future passenger demand, so observations are split by ordered **forecast month** to keep future periods out of training set and preserves test period as true unseen holdout.

===== WEEK 3 START =====

1. Environment and reusable project code

Week 3 builds on validated Week 2 ingestion pipeline stored in `Src/week2_src_t100_ingest_explore.py`. Week 3 logic is stored in `Src/week3_src_eda_split.py`.

This cell:

- Finds project root automatically.
- Imports Week 3 module.
- Confirms paths used for generated tables and figures.

```
In [16]: from pathlib import Path
from IPython.display import display, Image
import importlib.util
import pandas as pd

def find_project_root(start=None):
    start = Path(start or Path.cwd()).resolve()
    for path in [start, *start.parents]:
        if (path / "Data").exists() and (path / "Src").exists() and (path / "Notebooks").exists():
            return path
    raise FileNotFoundError("Could not locate the project root.")

PROJECT_ROOT = find_project_root()
REPORTS_DIR = PROJECT_ROOT / "Reports"
WEEK3_REPORTS_DIR = REPORTS_DIR / "week3"
WEEK3_TABLES_DIR = WEEK3_REPORTS_DIR / "week3_tables"
WEEK3_FIGURES_DIR = WEEK3_REPORTS_DIR / "week3_figures"
SRC_DIR = PROJECT_ROOT / "Src"

module_path = SRC_DIR / "week3_src_eda_split.py"
spec = importlib.util.spec_from_file_location("week3_eda", module_path)
week3 = importlib.util.module_from_spec(spec)
spec.loader.exec_module(week3)

print("Project root:", PROJECT_ROOT)
print("Week 3 source:", module_path)
print("Reports directory:", REPORTS_DIR)
```

2. Rebuild Week 2 Analytical Table

Week 2 pipeline:

- Reads annual BTS ZIP archives
- Filters project scope to scheduled passenger service
- Aggregates to **carrier × origin × destination × month** level
- Constructs next-month passenger target

```
In [17]: monthly = week3.load_week2_monthly(PROJECT_ROOT)

print(f"Carrier-route-month rows: {len(monthly):,}")
print(f"Columns: {monthly.shape[1]:,}")
print(f"Reference period: {monthly['Period'].min():%Y-%m} through {monthly['Period'].max():%Y-%m}")
print(f"Rows with observed next-month target: {monthly[week3.TARGET].notna().sum():,}")
print(f"Rows without observed next-month target: {monthly[week3.TARGET].isna().sum():,}")
```

Carrier-route-month rows: 1,882,779
Columns: 24
Reference period: 2015-01 through 2026-05
Rows with observed next-month target: 1,566,289
Rows without observed next-month target: 316,490

3. Determine Model Eligibility Before Splitting

Rows without observed `TargetPassengersNextMonth` cannot be used for supervised regression because outcome is unknown. They are **excluded rather than imputed**.

```
In [18]: model_data, eligibility_summary = week3.prepare_model_data(monthly)
display(eligibility_summary)
```

	Metric	Value
0	Carrier-route-month rows before target eligibi...	1.882779e+06
1	Rows with missing reference month	0.000000e+00
2	Rows with missing next-month target	3.164900e+05
3	Rows with negative next-month target	0.000000e+00
4	Rows eligible for Week 3 splitting/modeling	1.566289e+06
5	Percent of rows eligible	8.319030e+01
6	Percent missing next-month target	1.680970e+01

4. Chronological 70% / 15% / 15% Partition

Split is based on ordered unique **forecast months**:

- **Training**: earliest 70% of forecast months; used for EDA and later model fitting.
- **Validation**: next 15%; reserved for later model comparison/tuning.
- **Test**: most recent 15% (subject to whole-month rounding); untouched until final model evaluation.

All rows belonging to same forecast month remain in same partition. This prevents calendar month from being represented simultaneously in training / future holdout.

```
In [19]: (
    train_df,
    validation_df,
    test_df,
    split_calendar,
    split_summary,
) = week3.chronological_split(model_data)

display(split_summary)

print("Chronological integrity checks:")
print("  Training ends before validation starts:",
      train_df[week3.TARGET_TIME_COLUMN].max() < validation_df[week3.TARGET_TIME_COLUMN].min())
print("  Validation ends before test starts:",
      validation_df[week3.TARGET_TIME_COLUMN].max() < test_df[week3.TARGET_TIME_COLUMN].min())
print("  All eligible rows assigned:",
      len(train_df) + len(validation_df) + len(test_df) == len(model_data))
```

	Split	Rows	ForecastMonths	FirstReferenceMonth	LastReferenceMonth	FirstForecastMonth	LastForecastMonth	RowPercent	MonthPercent
0	Training	1062603	95	2015-01-01	2022-11-01	2015-02-01	2022-12-01	67.842078	69.852941
1	Validation	239395	20	2022-12-01	2024-07-01	2023-01-01	2024-08-01	15.284216	14.705882
2	Test	264291	21	2024-08-01	2026-04-01	2024-09-01	2026-05-01	16.873706	15.441176

Chronological integrity checks:
Training ends before validation starts: True
Validation ends before test starts: True
All eligible rows assigned: True

5. Training-only EDA

From this point forward, **no distributional EDA is conducted on validation or test observations**. The following analysis uses `train_df` only:

- structural/profile checks
- missingness and data types
- logical-value and continuity checks
- descriptive statistics, skewness, and kurtosis
- target distribution
- numeric predictor correlations
- calendar seasonality
- monthly time trend
- categorical cardinality

The validation and test partitions remain reserved for later modeling weeks.

```
In [20]: (
    training_profile,
    missingness,
    data_issues,
```

```

    numeric_summary,
    target_correlations,
    seasonality,
    monthly_trend,
    categorical_profile,
    key_findings,
) = week3.build_training_eda(monthly, train_df)

display(training_profile)

```

	Metric	Value
0	Training rows	1062603
1	Training columns	26
2	First training forecast month	2015-02-01 00:00:00
3	Last training forecast month	2022-12-01 00:00:00
4	Training forecast months	95
5	Unique carriers in training	137
6	Unique directional routes in training	14095
7	Unique carrier-route series in training	35388

5.1 Missingness, cardinality, and data types

In [21]: `display(missingness)`

	Variable	MissingCount	MissingPercent	UniqueValues	DataType
0	LoadFactor	1413	0.132975	624318	float64
1	UniqueCarrier	20	0.001882	137	string
2	DepPerformed	0	0.000000	833	float64
3	DepScheduled	0	0.000000	900	float64
4	Dest	0	0.000000	806	string
5	Distance	0	0.000000	1995	float64
6	Origin	0	0.000000	802	string
7	Passengers	0	0.000000	44675	float64
8	Seats	0	0.000000	50632	float64
9	TargetPassengersNextMonth	0	0.000000	44697	float64

5.2 Data-quality and continuity checks

In [22]: `display(data_issues)`

	Issue	Count	PercentOfTrainingRows	Interpretation	RecommendedTreatment
0	Duplicate carrier-route-month keys	0	0.000000	Data-integrity check	Investigate duplicates before modeling; Week 2...
1	Negative values in TargetPassengersNextMonth	0	0.000000	Logically invalid if present	Trace back to source and correct/exclude only ...
2	Negative values in Passengers	0	0.000000	Logically invalid if present	Trace back to source and correct/exclude only ...
3	Negative values in Seats	0	0.000000	Logically invalid if present	Trace back to source and correct/exclude only ...
4	Negative values in DepScheduled	0	0.000000	Logically invalid if present	Trace back to source and correct/exclude only ...
5	Negative values in DepPerformed	0	0.000000	Logically invalid if present	Trace back to source and correct/exclude only ...
6	Negative values in Distance	0	0.000000	Logically invalid if present	Trace back to source and correct/exclude only ...
7	Positive passengers with zero/nonpositive seats	0	0.000000	Potential reporting inconsistency	Investigate before modeling; do not silently i...
8	Load factor below 0	0	0.000000	Logically invalid if present	Investigate source records.
9	Load factor above 1	38	0.003576	Potential aggregation/reporting inconsistency	Review before deciding whether to cap, correct...
10	Rows that follow a gap of more than one month ...	55165	5.191497	Unbalanced-panel continuity issue affecting 19...	Create lag/rolling features only when exact pr...
11	Rows with at least one missing candidate predi...	1433	0.134858	Missing-feature issue	Fit any imputation rules on training data only...

5.3 Numeric descriptive statistics

In [23]:

display(numeric_summary)

	Variable	count	mean	std	min	1%	5%	25%	50%	75%	95%	99%	max
0	Passengers	1062603.0	5065.178166	7738.886913	0.0	0.0	13.000000	792.000000	2507.000	5678.000000	20207.000000	39450.980000	97794.000000
1	Seats	1062603.0	6390.679652	9272.675308	0.0	9.0	54.000000	1266.000000	3420.000	7513.000000	24885.000000	46607.980000	111897.000000
2	DepScheduled	1062603.0	55.833898	65.448957	0.0	0.0	0.000000	13.000000	31.000	76.000000	180.000000	305.000000	2605.000000
3	DepPerformed	1062603.0	55.851902	61.836942	0.0	1.0	2.000000	17.000000	32.000	74.000000	175.000000	296.000000	1247.000000
4	Distance	1062603.0	733.079550	615.145679	0.0	12.0	46.000000	282.000000	594.000	1009.000000	2065.000000	2611.000000	6089.000000
5	LoadFactor	1061190.0	0.712431	0.227918	0.0	0.0	0.195122	0.625397	0.792	0.873975	0.938845	0.969162	1.218710
6	TargetPassengersNextMonth	1062603.0	5048.629952	7757.965492	0.0	0.0	12.000000	685.000000	2479.000	5685.000000	20224.000000	39475.000000	97794.000000

5.4 Numeric relationships with the next-month target

In [24]:

display(target_correlations)

	Predictor	PearsonCorrelationWithTarget	AbsoluteCorrelation
0	Passengers	0.976927	0.976927
1	Seats	0.963833	0.963833
2	DepPerformed	0.821657	0.821657
3	DepScheduled	0.793736	0.793736
4	LoadFactor	0.289594	0.289594
5	Distance	0.194469	0.194469

5.5 Calendar seasonality

In [25]:

display(seasonality)

	ForecastCalendarMonth	MeanTargetPassengers	MedianTargetPassengers	TotalTargetPassengers	Observations	MonthName
0	1	4303.097325	2000.0	352075120.0	81819	Jan
1	2	4369.013709	2137.0	393582600.0	90085	Feb
2	3	5103.214658	2534.0	457666497.0	89682	Mar
3	4	4734.427080	2087.0	416407065.0	87953	Apr
4	5	5314.610839	2600.0	454122867.0	85448	May
5	6	5373.090955	2667.0	472418276.0	87923	Jun
6	7	5448.567741	2761.0	502619477.0	92248	Jul
7	8	5186.494354	2538.0	488723363.0	94230	Aug
8	9	4877.262057	2443.0	436978294.0	89595	Sep
9	10	5538.891652	2887.0	478344222.0	86361	Oct
10	11	5186.062273	2640.0	453204796.0	87389	Nov
11	12	5102.333994	2553.0	458546756.0	89870	Dec

5.6 Categorical cardinality

In [26]:

display(categorical_profile)

	Variable	UniqueValues	MissingCount	MostFrequentValue	MostFrequentCount	MostFrequentPercent
0	UniqueCarrier	137	20	WN	125988	11.856545
1	Origin	802	0	ORD	44913	4.226696
2	Dest	806	0	ORD	45190	4.252764
3	Route	14095	0	LAS→LAX	773	0.072746

6. Save Week 3 artifacts and visualize the training results

The saved figures below use the training partition only, except the split-size chart, which simply documents partition sizes.

In [27]:

results = week3.Week3Results(
 monthly=monthly,
 model_data=model_data,
 train_df=train_df,
 validation_df=validation_df,

```

test_df=test_df,
split_calendar=split_calendar,
split_summary=split_summary,
eligibility_summary=eligibility_summary,
training_profile=training_profile,
missingness=missingness,
data_issues=data_issues,
numeric_summary=numeric_summary,
target_correlations=target_correlations,
seasonality=seasonality,
monthly_trend=monthly_trend,
categorical_profile=categorical_profile,
key_findings=key_findings,
preprocessing_recommendations=week3._preprocessing_recommendations(),
)

week3.save_outputs(results, PROJECT_ROOT)

print("Saved tables:", WEEK3_TABLES_DIR)
print("Saved figures:", WEEK3_FIGURES_DIR)

```

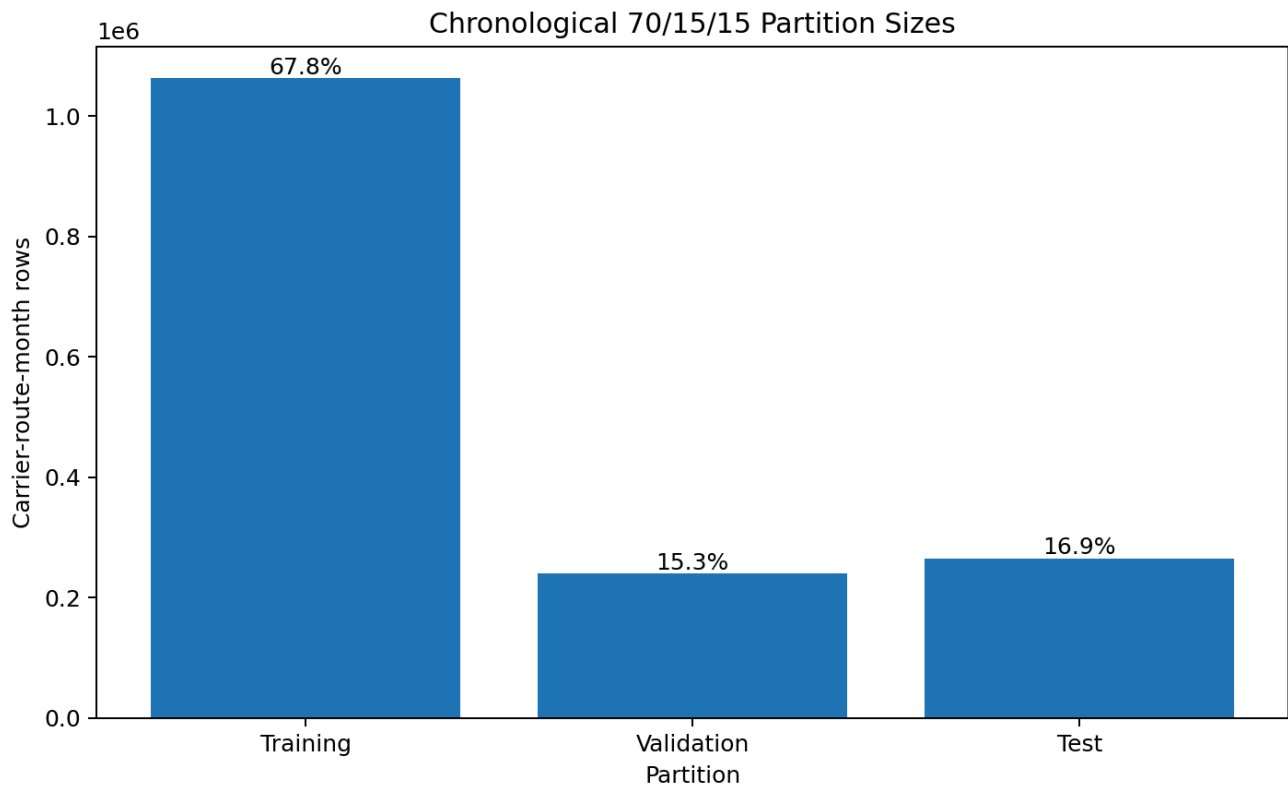
```

In [28]: figure_paths = [
    WEEK3_FIGURES_DIR / "week3_figure_split_sizes.png",
    WEEK3_FIGURES_DIR / "week3_figure_training_target_distribution.png",
    WEEK3_FIGURES_DIR / "week3_figure_training_demand_over_time.png",
    WEEK3_FIGURES_DIR / "week3_figure_training_seasonality.png",
    WEEK3_FIGURES_DIR / "week3_figure_training_target_correlations.png",
]

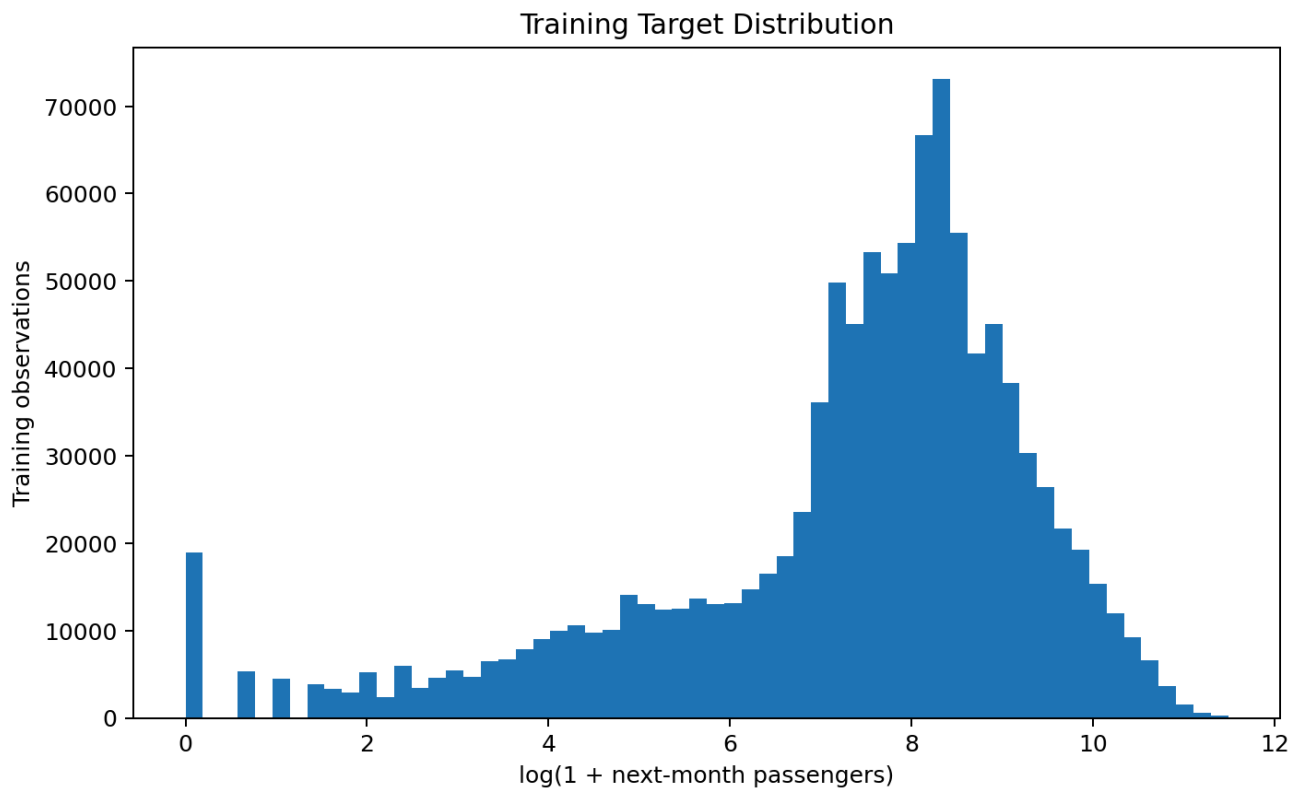
for path in figure_paths:
    if path.exists():
        print(path.name)
        display(Image(filename=str(path)))

```

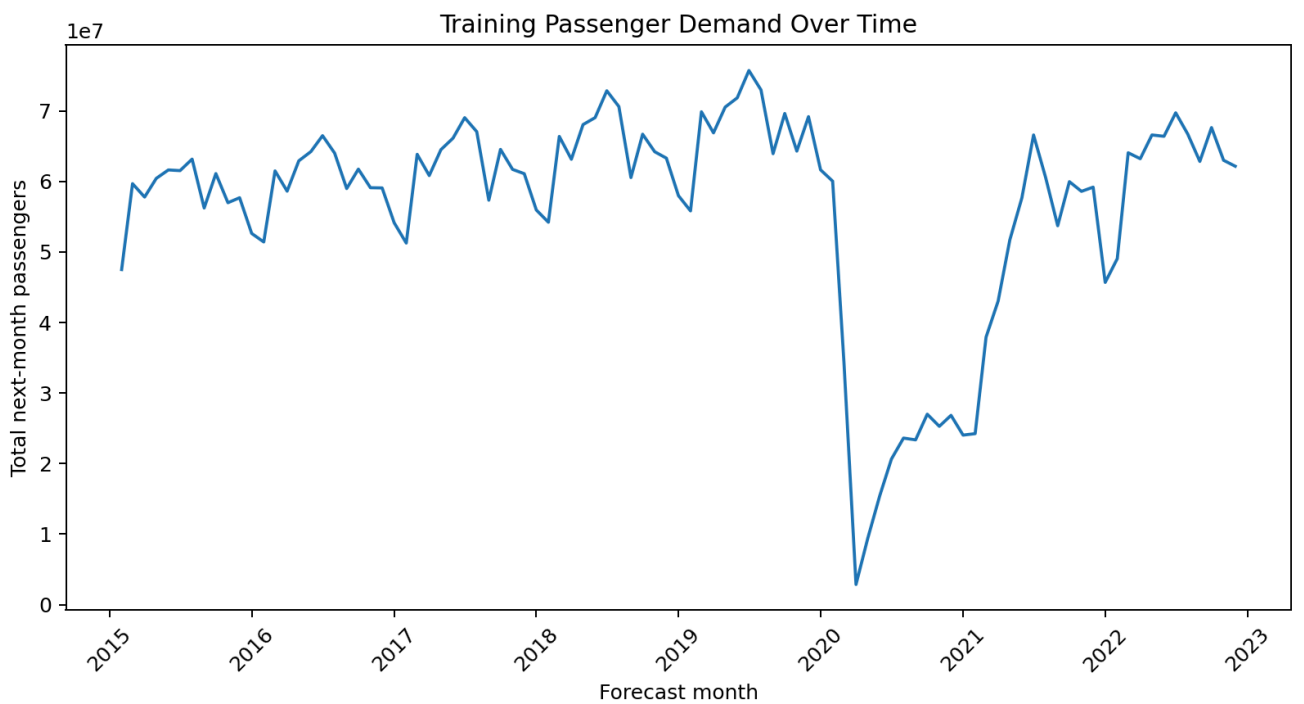
week3_figure_split_sizes.png



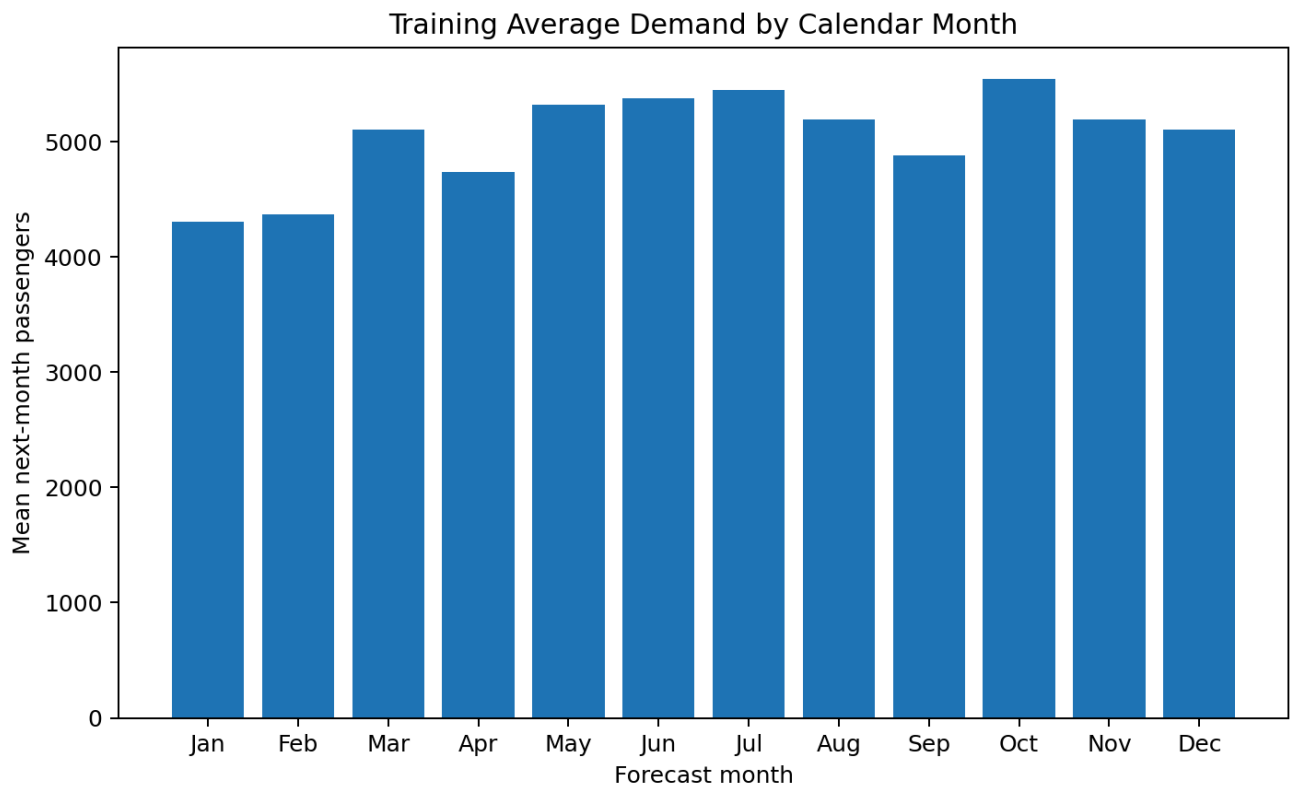
week3_figure_training_target_distribution.png



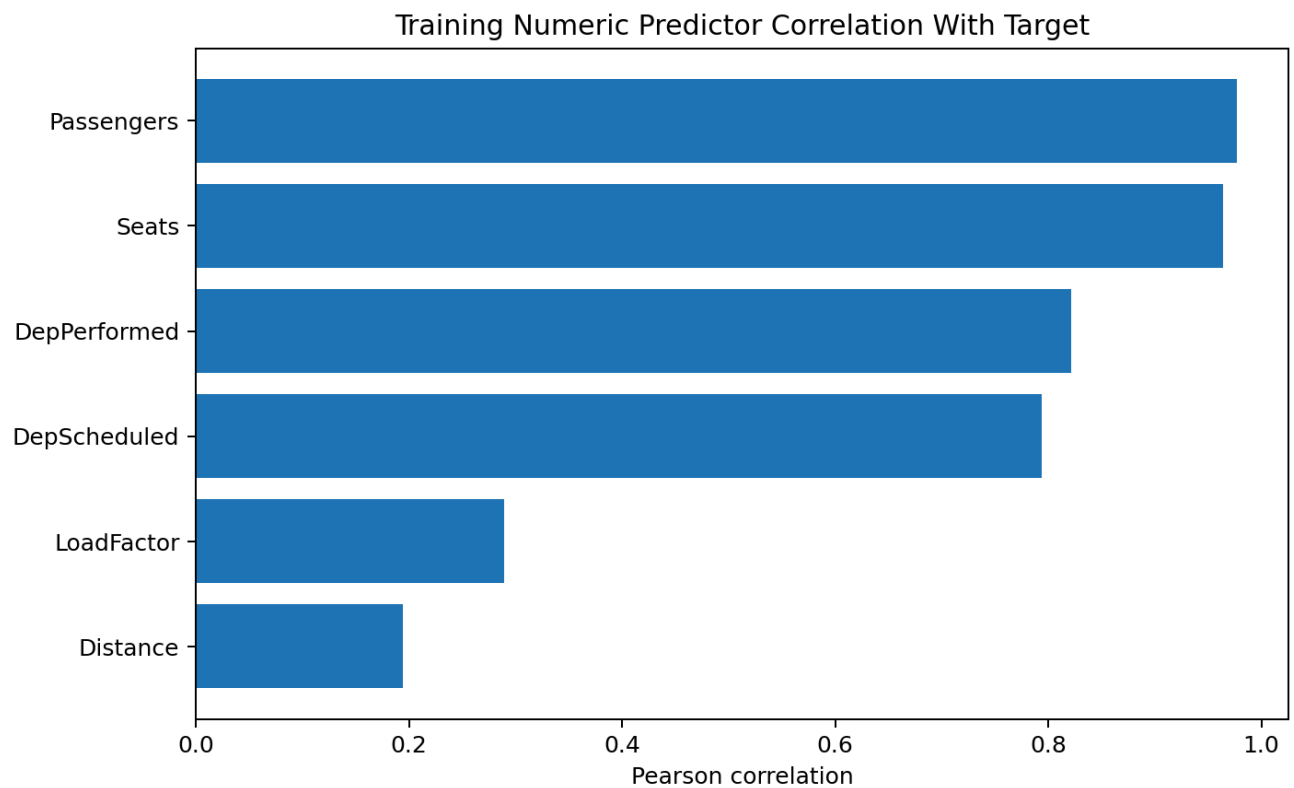
week3_figure_training_demand_over_time.png



week3_figure_training_seasonality.png



week3_figure_training_target_correlations.png



7. Key findings and preprocessing recommendations

```
In [29]: display(key_findings)
display(results.preprocessing_recommendations)
```

	Finding	Evidence
0	Structural target availability	316,490 of 1,882,779 carrier-route-month rows ...
1	Target distribution	In training, target mean is 5,048.63, median i...
2	Strongest numeric relationship	Passengers has largest absolute Pearson correl...
3	Seasonality	Average next-month demand is highest in Oct (5...
4	Time-regime variation	lowest aggregate training demand occurs in 202...
5	Unbalanced carrier-route panel	55,165 training observations follow a gap long...

Priority	Recommendation	Rationale
0	1 Preserve chronological month-based split and n...	project predicts future demand; temporal leaka...
1	2 Do not impute missing target values; exclude r...	supervised outcome is unknown, not merely a mi...
2	3 Fit all predictor imputation, encoding, scalin...	Using validation/test distributions to design ...
3	4 Represent carrier/origin/destination categoric...	These are high-cardinality identifiers and can...
4	5 Consider log1p target modeling, robust losses,...	Route-month passenger demand is expected to be...
5	6 Engineer lags/rolling features only from infor...	carrier-route panel is unbalanced; naive shift...
6	7 Consider calendar seasonality and a pandemic/r...	time series spans materially different demand ...

8. Week 3 completion checks

The analysis is considered complete when:

- every eligible observation belongs to exactly one chronological partition,
- validation/test months occur strictly after the training months,
- EDA tables/figures are based on training data only,
- the notebook runs from top to bottom without errors,
- report-ready tables/figures exist.

```
In [30]: # Confirm strict chronological separation among the partitions.
assert (
    train_df[week3.TARGET_TIME_COLUMN].max()
    < validation_df[week3.TARGET_TIME_COLUMN].min()
), "Training and validation periods overlap."

assert (
    validation_df[week3.TARGET_TIME_COLUMN].max()
    < test_df[week3.TARGET_TIME_COLUMN].min()
), "Validation and test periods overlap."

# Confirm every modeling observation belongs to exactly one partition.
assert (
    len(train_df) + len(validation_df) + len(test_df)
    == len(model_data)
), "Partition row counts do not sum to the modeling dataset."

# Confirm critical Week 3 outputs were generated.
assert (
    WEEK3_TABLES_DIR / "week3_results_summary.json"
).exists(), "Week 3 results summary was not generated."

assert (
    WEEK3_FIGURES_DIR / "week3_figure_training_target_distribution.png"
).exists(), "Week 3 target-distribution figure was not generated."

print("All Week 3 checks passed.")
print("==== WEEK 3 END =====")
```

All Week 3 checks passed.
==== WEEK 3 END =====